

PRACTICAL-3

Aim:

To implement Heap Sort using the Heapify technique in C++ and analyze its time complexity

input : An Array A of n elements.
output : sorted array in ascending order.
steps :
Step 1: Start
Step 2: Read the array A and its size n .
Step 3: Build a max heap using $\text{heapify}()$
Step 4: Swap the first element with the last element.
Step 5: Reduce the heap size by 1.
Step 6: Apply $\text{heapify}()$ to maintain the max heap.
Step 7: Repeat step 4-6 until the heap size becomes 1.
Step 8: Display the sorted array.
Step 9: Stop.

Program:

1. heap sort:

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;

    int left = 2 * i + 1;

    int right = 2 * i + 2;

    if(left < n && arr[left] > arr[largest])

        largest = left;

    if(right < n && arr[right] > arr[largest])

        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);

        heapify(arr, n, largest);
    }
}
```

```
void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};

    int n = 6;

    cout << "Original Array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    heapSort(arr, n);

    cout << "\nSorted Array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

Output:

Output

Original Array: 12 11 13 5 6 7

Sorted Array: 5 6 7 11 12 13

=== Code Execution Successful ===

Time Complexity:

Actual optimized Build - Max heap - takes $O(n)$
 $n = \text{nodes}$
for $n-1$ swaps, each heapify can takes almost
 $\log n$ comparisons
 $(n-1) O(\log n)$
 $= (n-1) (\log n)$
 $= \text{wct}(n \log n)$
Best case:- $T(n) = O(n) + (n-1) (O(\log n))$
 $= O(n) + O(n \log n)$
 $= n \log n > 0$
 $= n \log n$
Avg case:- $T(n) = O(n) + (n-1) (O(\log n))$
 $= O(n) + O(n \log n)$
 $= O(n \log n)$
worst case:- height of heap $= h = \log n$
1 heapify $= O(\log n)$
for $n-1$ heapify $= (n-1) O(\log n)$
 $= O(n \log n)$
worst case $= O(n \log n)$

Conclusion:

Heap Sort was successfully implemented using the Heapify technique. The array was sorted in ascending order with $O(n \log n)$ time complexity.